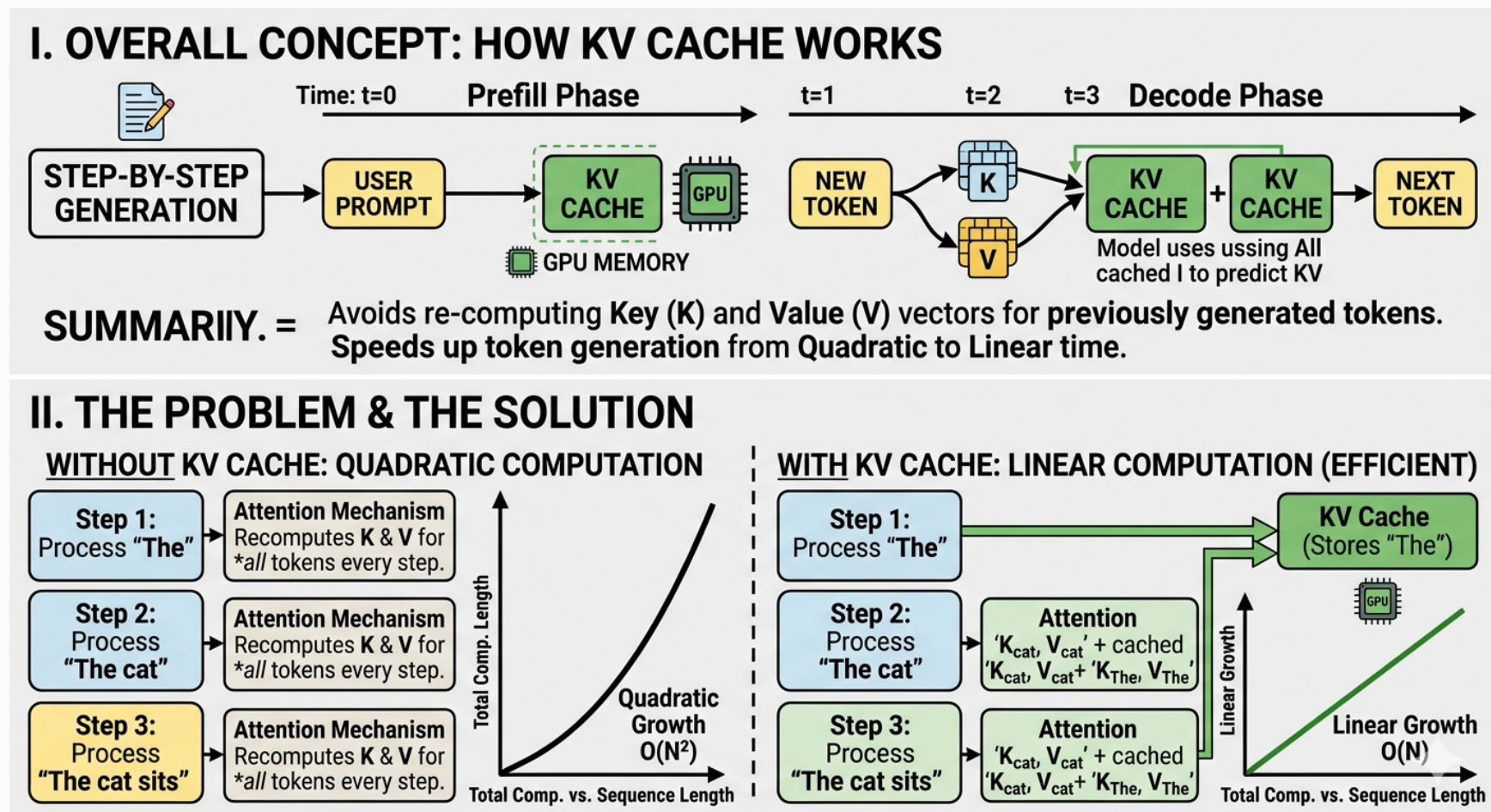


FetchLess: Profiling and Expanding on State of the Art KV Cache Retrieval Systems

Aditya Gupta | Andrew Koulogeorge | Steven Kolawole | 15-618 Spring 2026 | Carnegie Mellon University

Background: KV Cache & Reasoning LLMs



Reasoning Models Models like DeepSeek-R1/o1 generate *thousands* of intermediate tokens before producing an answer, ballooning KV cache size.

Two Solutions for Large KV Cache Management:

Eviction: Permanently drop “unimportant” tokens

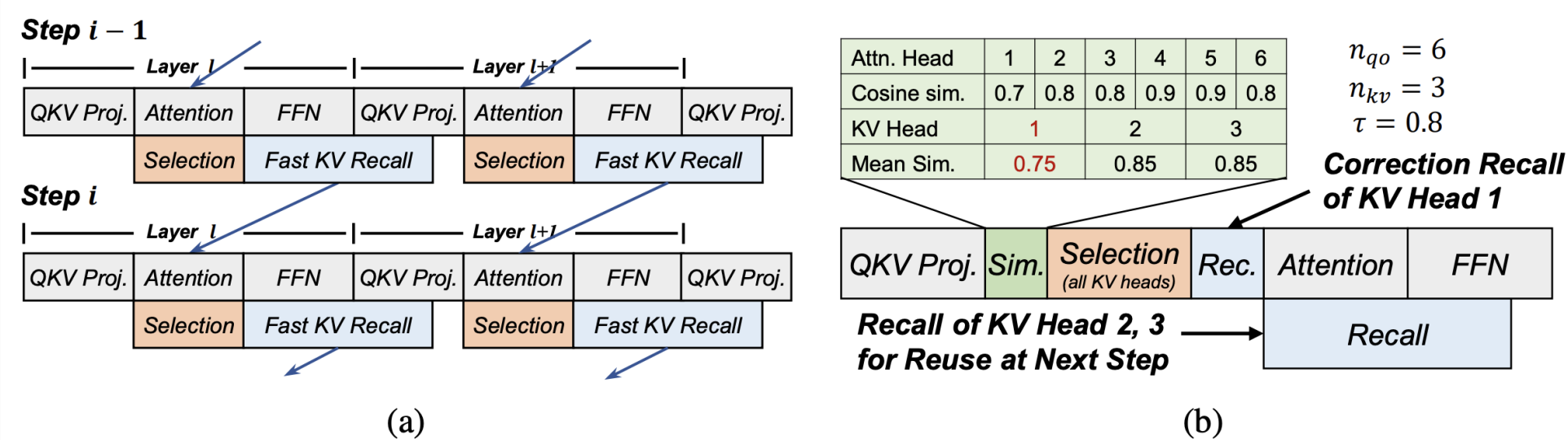
Retrieval: Offload the cache to CPU and dynamically fetch the relevant subset each decode step.

FreeKV: SOTA KV Cache Retrieval

Observation. Adjacent decode-step queries have high cosine similarity. If two queries are highly aligned, they will attend to similar KV pages.

Llama-3.1-8B, layer 31	DeepSeek-R1-14B, layer 16
0.97 0.96 0.96 0.97	0.94 0.94 0.94 0.93 0.95
0.98 0.95 0.96 0.97	0.92 0.91 0.95 0.94 0.93
0.97 0.96 0.96 0.94	0.93 0.85 0.86 0.92 0.92
0.97 0.96 0.92 0.97	0.86 0.88 0.90 0.89 0.88
0.96 0.95 0.96 0.96	0.84 0.86 0.86 0.88 0.89
0.98 0.98 0.98 0.90	0.91 0.86 0.91 0.91 0.86
0.96 0.96 0.97 0.96	0.91 0.94 0.93 0.94 0.89
0.96 0.96 0.96 0.98	0.86 0.92 0.90 0.89 0.90

Algorithm. Retrieve KV pages from CPU only when current query is different enough from previous queries, otherwise reuse KV pages from the previous time step



(Left) FreeKV when cosine sim all above threshold. (Right) FreeKV when at least 1 head below threshold. Selection and Recall brought onto critical path

Key Results

13× generation speedup compared to prior retrieval methods with lossless accuracy.

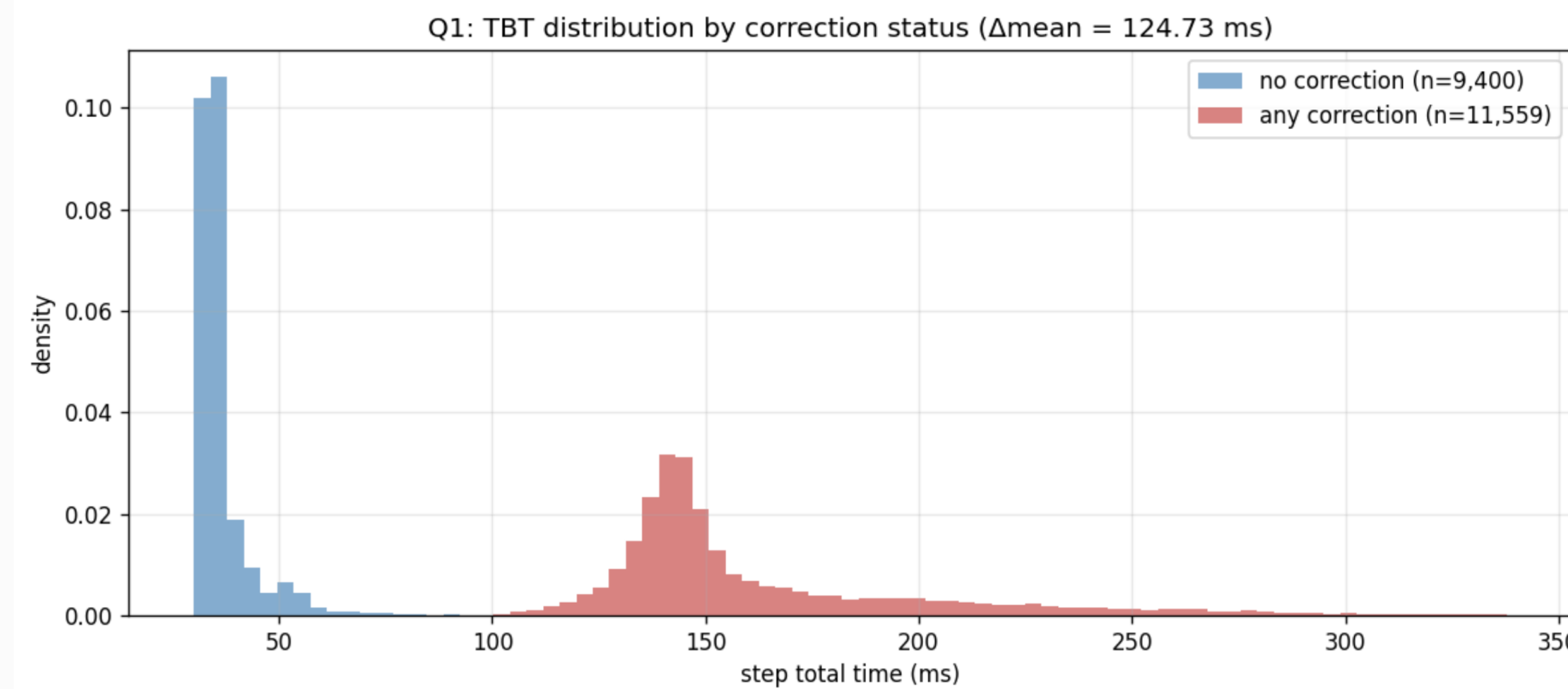
Lower cosine similarity threshold results in fetching pages less often = worse down stream performance.

Deeper Analysis of FreeKV

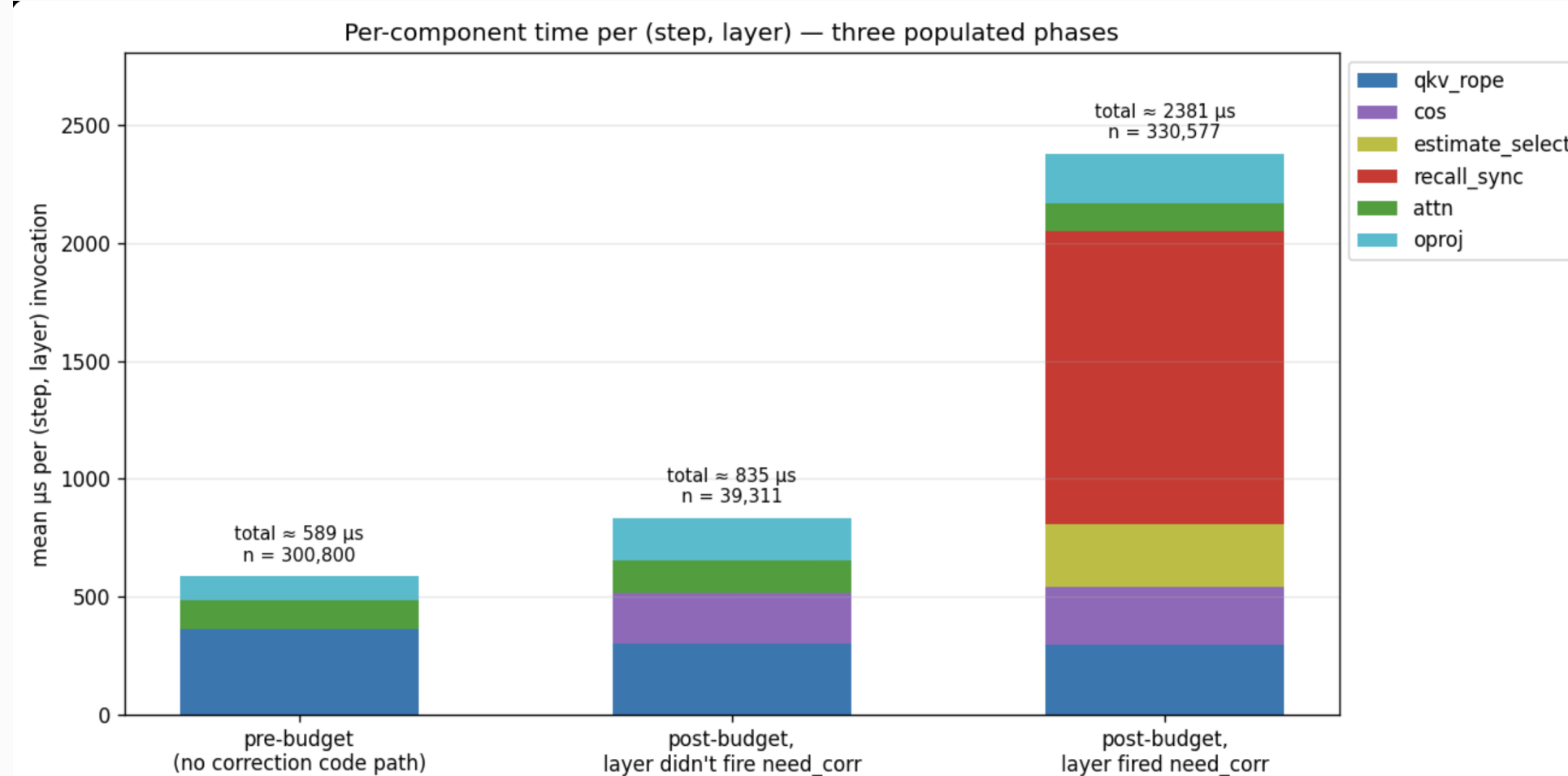
Experimentation Configuration: DeepSeek-R1-Distill-LLaMA-8B on AIME 2024 with one A100 40GB GPU

Q1: When retrieval triggers, how expensive is it for the critical path?

Selection and recall on the critical path: [$\sim 2.5\times$] increase to generation latency

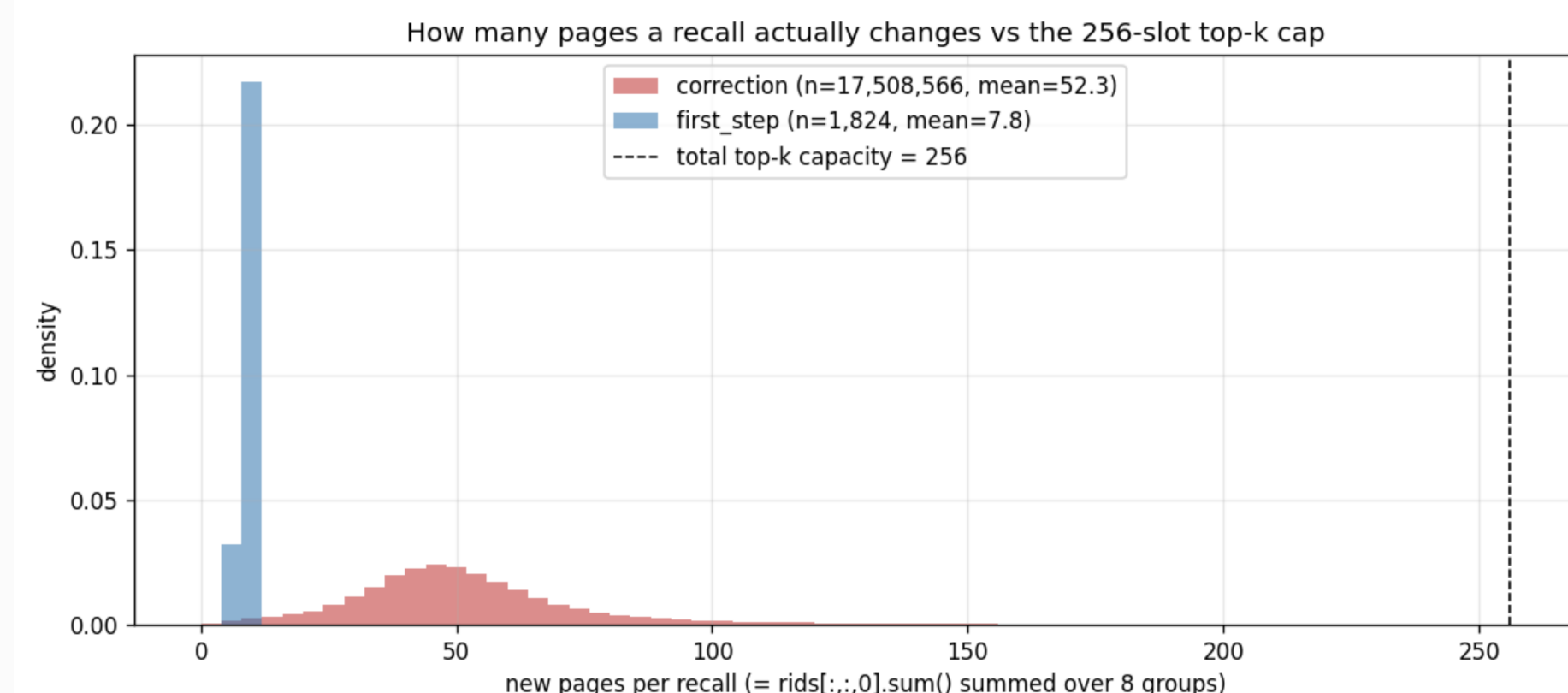


When retrieval is on the critical path, similarity, recall accounts for largest chunk of token generation latency



Q2: When retrieval triggers, how many of the fetched pages are actually new?

Only [$\sim 20\%$] of the pages returned by retrieval are *not* already resident in the on-GPU page buffer

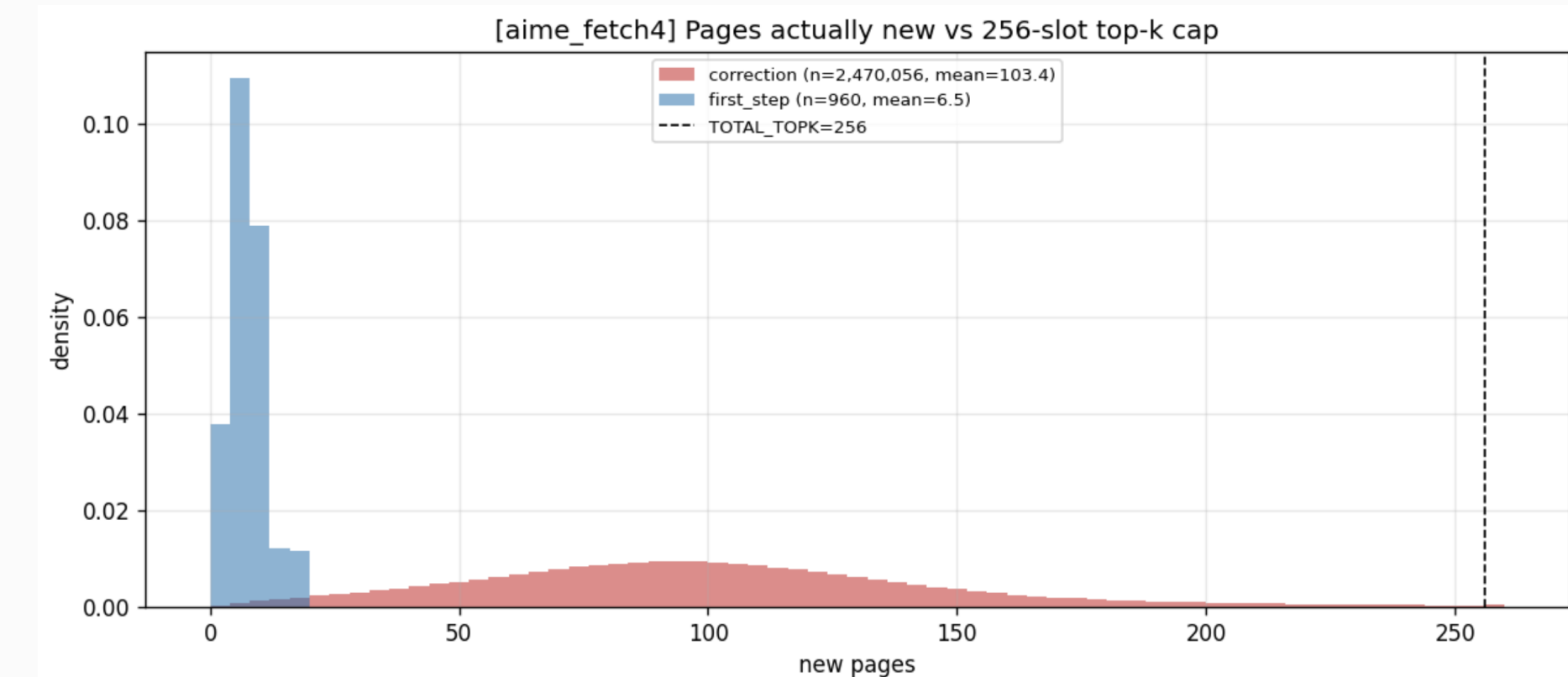
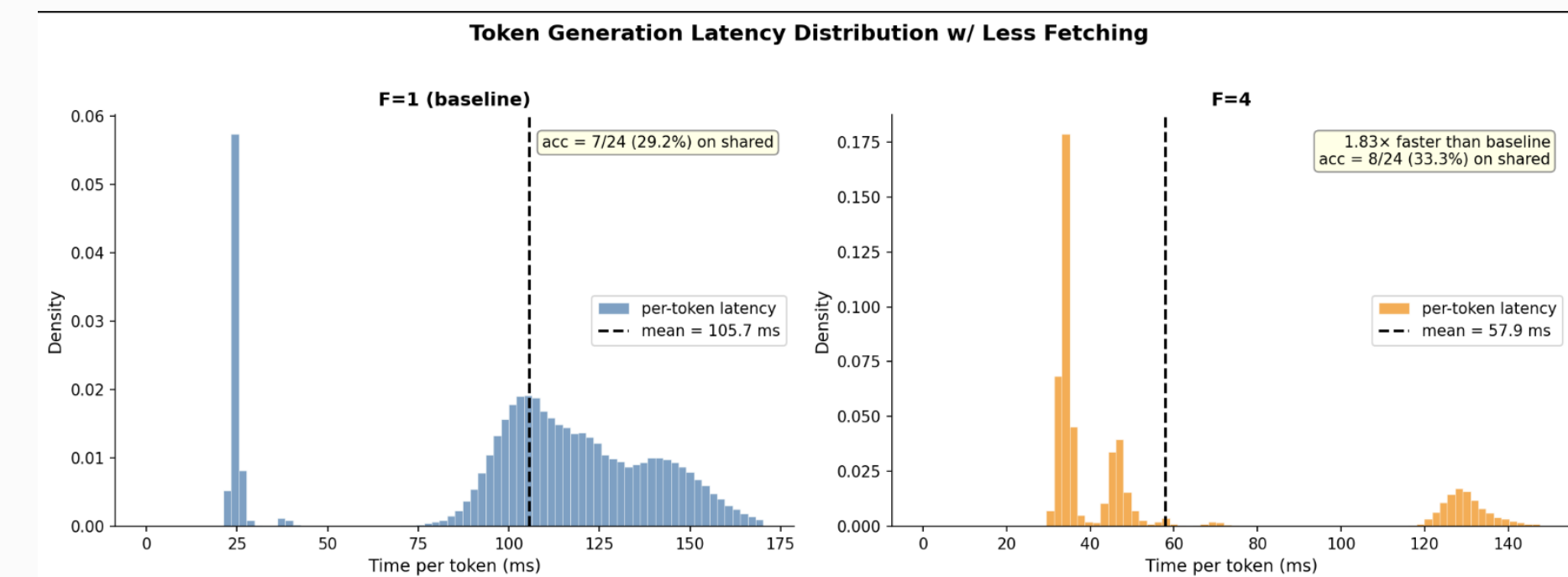


FetchLess Experiment

Research question. Motivated by the analysis in Column 2 — redundant fetches and high retrieval-logic cost — can we *trigger fewer fetches* from the KV cache to obtain end-to-end inference speedups without harming task accuracy?

Method: token-level fetch skipping. Standard FreeKV runs its retrieval pipeline at every layer of every decode step. **FetchLess** skips the fetch logic for an entire token — across *all* transformer layers — on a configurable schedule (every F -th token), reusing the previously selected page set in between.

Results: Near $2\times$ latency speedup with no performance drop on AIME



Future Work

(1) Broader experimental sweep. Extend FetchLess beyond a single model/dataset:

Additional reasoning models (Qwen-QwQ, o1-style distills) and benchmarks (MATH-500, GPQA, LiveCodeBench).

Full sweep of F to characterize the accuracy/latency pareto frontier.

(2) Thought-type-aware retrieval. Our preliminary analysis shows the cosine-similarity distribution is sharply different on *transition* tokens (“wait”, “actually”) vs. regular reasoning tokens — these are exactly where FreeKV’s correction check fires.

